

# Especificação do Projeto

Para fins didáticos, o melhor é implementar o **IBM Model 1** como um pequeno projeto Rust que mostre claramente a passagem:

**corpus paralelo**  $\rightarrow$  **vocabulários**  $\rightarrow$  **tabela**  $t(f|e)$   $\rightarrow$  **EM**  $\rightarrow$  **alinhamentos prováveis**

O artigo define o Model 1 como um modelo que estima probabilidades de tradução  $t(f|e)$  e assume que cada palavra francesa pode se alinhar uniformemente a qualquer palavra inglesa da sentença, incluindo o elemento vazio  $e_0$ ; essa simplicidade permite calcular as somas sobre alinhamentos de forma eficiente, sem enumerar todos os alinhamentos possíveis.

## 1. Objetivo didático do projeto

O objetivo não deve ser criar um tradutor completo. O foco deve ser:

1. Ler pares de sentenças paralelas.
2. Tokenizar inglês e francês.
3. Criar vocabulários internos.
4. Inicializar  $t(f|e)$ .
5. Executar o algoritmo EM.
6. Exibir a tabela de probabilidades aprendidas.
7. Gerar alinhamentos palavra a palavra por maior probabilidade.

Exemplo de saída desejada:

$t(\text{maison} \mid \text{house}) = 0.82$

$t(\text{livre} \mid \text{book}) = 0.79$

$t(\text{le} \mid \text{the}) = 0.55$

$t(\text{la} \mid \text{the}) = 0.42$

E alinhamento:

the house

le maison

le -> the  
maison -> house

## 2. Estrutura recomendada do projeto Rust

Uma estrutura didática adequada seria:

```
ibm_model_1_rust/  
├── Cargo.toml  
└── src/  
    ├── main.rs  
    ├── corpus.rs  
    ├── vocab.rs  
    ├── model.rs  
    ├── trainer.rs  
    └── aligner.rs
```

### Responsabilidade de cada módulo

| Módulo    | Responsabilidade                  |
|-----------|-----------------------------------|
| corpus.rs | Ler pares de sentenças paralelas  |
| vocab.rs  | Mapear palavras para IDs inteiros |
| model.rs  | Armazenar `t(f                    |

```

    tr          Executar EM
    ainer.
    rs

    al          Gerar alinhamento final
    igner.
    rs

    ma          Orquestrar o fluxo e exibir
    in.rs      resultados

```

Para uma primeira versão didática, também é aceitável colocar tudo em `main.rs`. Mas, como você quer demonstrar design, a separação em módulos é melhor.

### 3. Representação dos dados

#### Opção didática recomendada

Usar IDs inteiros para palavras:

```

type TokenId = usize;

struct SentencePair {
    source: Vec<TokenId>, // inglês: e
    target: Vec<TokenId>, // francês: f
}

```

No contexto do artigo:

```

source = e = sentença inglesa
target = f = sentença francesa

```

O Model 1 calcula:

$$t(f | e)$$

ou seja:

probabilidade de uma palavra francesa  $f$  dado uma palavra inglesa  $e$

### Por que usar IDs e não **String** diretamente?

Porque isso facilita:

- indexação;
- uso de **Vec**;
- comparação rápida;
- tabelas de probabilidade;
- expansão futura para modelos maiores.

Para fins didáticos, o vocabulário pode ser:

```
struct Vocabulary {  
    word_to_id: HashMap<String, TokenId>,  
    id_to_word: Vec<String>,  
}
```

## 4. Tratamento do token **NULL**

O IBM Model 1 usa um elemento especial, normalmente chamado de **NULL**, **empty cept** ou **e<sub>0</sub>**. Ele representa palavras francesas que não têm uma palavra inglesa correspondente direta.

No código:

```
const NULL_TOKEN: &str = "<NULL>";
```

Cada sentença fonte deve ser processada assim:

```
["<NULL>", "the", "house"]
```

Isso corresponde ao fato de que, no Model 1, cada palavra francesa pode se alinhar a qualquer posição inglesa de 0 até 1. O artigo explicita essa ideia ao reservar a posição zero para o empty cept.

## 5. Representação da tabela $t(f|e)$

Aqui há três alternativas de design.

**Alternativa A** — `HashMap<(e, f), f64>`

Mais direta:

```
HashMap<(TokenId, TokenId), f64>
```

Vantagem: simples de entender.

Desvantagem: menos organizada para normalizar por  $e$ .

**Alternativa B** — `HashMap<TokenId, HashMap<TokenId, f64>>`

Mais didática para  $t(f|e)$ :

```
HashMap<TokenId, HashMap<TokenId, f64>>
```

Leitura natural:

```
t[e][f]
```

Vantagem: combina bem com a fórmula  $t(f|e)$ .

Desvantagem: mais verboso em Rust por causa de `entry`, `get`, `unwrap_or`.

**Alternativa C** — matriz densa `Vec<Vec<f64>>`

Exemplo:

```
Vec<Vec<f64>>
```

Vantagem: rápida e simples para corpus pequeno.

Desvantagem: desperdiça memória se o vocabulário for grande.

### **Recomendação**

Para demonstração didática:

`HashMap<TokenId, HashMap<TokenId, f64>>`

Para uma versão otimizada posterior:

`Vec<Vec<f64>>`

ou uma estrutura esparsa.

## **6. Inicialização de $t(f|e)$**

Antes do EM, você precisa inicializar as probabilidades.

A forma didática é:

1. Para cada palavra inglesa **e**, descobrir quais palavras francesas **f** aparecem em sentenças paralelas com ela.
2. Distribuir probabilidade uniforme entre essas palavras francesas.

Exemplo:

house aparece em sentenças com:  
maison, la

Então:

$t(\text{maison} | \text{house}) = 0.5$

$t(\text{la} | \text{house}) = 0.5$

Isso é melhor do que inicializar sobre todo o vocabulário francês, porque reduz ruído e custo.

## 7. Treinamento EM

O centro do código estará no `trainer.rs`.

A cada iteração:

### **E-step — calcular contagens esperadas**

Para cada par de sentenças (`e_sentence`, `f_sentence`):

Para cada palavra francesa `f_j`:

1. Calcular o denominador:

$$\text{total\_s}(f_j) = \sum_i t(f_j | e_i)$$

2. Para cada palavra inglesa `e_i`, incluindo `<NULL>`:

$$\text{count}(e_i, f_j) += t(f_j | e_i) / \text{total\_s}(f_j)$$

$$\text{total}(e_i) += t(f_j | e_i) / \text{total\_s}(f_j)$$

Essa é a forma computacional da ideia das equações (12), (14) e (17): como o alinhamento real não é observado, o modelo distribui uma “responsabilidade probabilística” entre as possíveis palavras inglesas que poderiam ter gerado cada palavra francesa.

### **M-step — atualizar $t(f|e)$**

Depois de percorrer o corpus:

$$t(f | e) = \text{count}(e, f) / \text{total}(e)$$

Isso corresponde à normalização final: para cada palavra inglesa `e`, as probabilidades das palavras francesas associadas devem somar 1.

## 8. Esqueleto do algoritmo em Rust

Um pseudo-Rust didático ficaria assim:

```

    for iteration in 0..num_iterations {
        let mut count: HashMap<TokenId, HashMap<TokenId, f64>> =
HashMap::new();
        let mut total: HashMap<TokenId, f64> = HashMap::new();

        for pair in &corpus {
            for &f in &pair.target {
                let mut denom = 0.0;

                for &e in &pair.source {
                    denom += model.prob(e, f);
                }

                for &e in &pair.source {
                    let delta = model.prob(e, f) / denom;

                    *count
                        .entry(e)
                        .or_default()
                        .entry(f)
                        .or_insert(0.0) += delta;

                    *total
                        .entry(e)
                        .or_insert(0.0) += delta;
                }
            }
        }

        for (e, f_counts) in count {
            for (f, c) in f_counts {
                let new_prob = c / total[&e];

```



```

        model.set_prob(e, f, new_prob);
    }
}
}

```

Esse trecho é a implementação central do Model 1.

## 9. Geração de alinhamentos

Depois do treinamento, para alinhar uma sentença:

Para cada palavra francesa  $f_j$ , escolher a palavra inglesa  $e_i$  com maior  $t(f_j|e_i)$ :

```

fn align(pair: &SentencePair, model: &TranslationTable) -> Vec<(TokenId,
TokenId)> {
    let mut links = Vec::new();

    for &f in &pair.target {
        let mut best_e = pair.source[0];
        let mut best_score = 0.0;

        for &e in &pair.source {
            let score = model.prob(e, f);

            if score > best_score {
                best_score = score;
                best_e = e;
            }
        }

        links.push((best_e, f));
    }
}

```

```
links  
}
```

Isso não é a tradução em si. É o **alinhamento mais provável palavra a palavra** segundo a tabela aprendida.

## 10. Entrada de dados para a versão didática

Use um arquivo simples **TSV**:

|           |            |
|-----------|------------|
| the house | la maison  |
| the book  | le livre   |
| a house   | une maison |
| my book   | mon livre  |

No código:

coluna 1 = inglês  
coluna 2 = francês

Evite, no começo:

- pontuação complexa;
- letras maiúsculas;
- frases muito longas;
- expressões idiomáticas;
- múltiplas traduções por frase.

## 11. Design didático recomendado

A melhor arquitetura para ensino seria:

```
struct IBMModel1 {  
    table: TranslationTable,  
    iterations: usize,
```

```

}

impl IBMModel1 {
    fn train(&mut self, corpus: &[SentencePair]) { ... }

    fn probability(&self, e: TokenId, f: TokenId) -> f64 { ... }

    fn align(&self, pair: &SentencePair) -> Vec<AlignmentLink> { ... }
}

```

E:

```

struct AlignmentLink {
    source: TokenId,
    target: TokenId,
    score: f64,
}

```

Assim, o aluno entende que o “modelo” contém:

parâmetros aprendidos = tabela  $t(f|e)$

e que o treinamento é o processo que ajusta essa tabela.

## 12. Alternativas de design

### Design 1 — versão pedagógica mínima

Tudo em memória, **HashMap**, corpus pequeno.

Vantagens:

- fácil de explicar;
- código curto;
- bom para sala de aula;

- permite imprimir cada etapa do EM.

Indicado para: demonstração conceitual.

### **Design 2 — versão intermediária**

Separar módulos, usar IDs, salvar modelo em JSON.

Vantagens:

- mais organizado;
- permite reuso;
- permite testes unitários;
- já parece um projeto real.

Indicado para: material didático mais robusto.

### **Design 3 — versão performática**

Usar:

- `Vec<Vec<f64>>` ou matriz esparsa;
- paralelismo com `rayon`;
- leitura streaming;
- corpus maior;
- serialização binária.

Vantagens:

- mais rápido;
- mais próximo de aplicações reais.

Desvantagem: atrapalha a explicação inicial.

Indicado para: segunda etapa do curso.

## **13. Testes unitários recomendados**

Crie testes simples para:

1. Verificar se o vocabulário cria IDs corretamente.
2. Verificar se `<NULL>` está na sentença fonte.
3. Verificar se  $\sum_f t(f|e) = 1$ .
4. Verificar se as probabilidades mudam após uma iteração.

5. Verificar se **maison** se alinha com **house** em um corpus controlado.

Exemplo conceitual:

```
assert!((sum_probs_for_e - 1.0).abs() < 1e-9);
```

## 14. Fluxo didático de execução

O programa poderia rodar assim:

```
cargo run -- --corpus data/toy.tsv --iterations 10
```

Saída:

Iteração 1

$t(\text{maison} \mid \text{house}) = 0.42$

$t(\text{livre} \mid \text{book}) = 0.39$

Iteração 10

$t(\text{maison} \mid \text{house}) = 0.86$

$t(\text{livre} \mid \text{book}) = 0.83$

Alinhamento:

la -> the

maison -> house

## 15. O que não implementar no primeiro momento

Para manter o foco didático, eu evitaria inicialmente:

- Modelos IBM 2 a 5;
- cálculo completo de  $\text{Pr}(f|e)$  com  $\epsilon$ ;
- busca por tradução;
- smoothing avançado;
- corpus grande;

- paralelismo;
- normalização em log-space;
- interface gráfica.

O ponto central é mostrar como uma tabela  $t(f|e)$  emerge de dados paralelos por EM.

## Síntese

A implementação didática em Rust deve tratar o IBM Model 1 como um **modelo probabilístico treinável**, não como uma rede neural. O núcleo do projeto é uma tabela de probabilidades  $t(f|e)$  atualizada iterativamente por EM. O design mais adequado para ensino é usar corpus pequeno, vocabulários com IDs, `HashMap<TokenId, HashMap<TokenId, f64>>`, módulos separados e saída textual mostrando as probabilidades após cada iteração.